

Programacion Orientada a Objetos

Segundo Ciclo

Fabricio Xiloj

Carnet: 262140



ANÁLISIS: CASO MISTERIOSO DE LA AGENCIA DE DETECTIVES

1. Requisitos Funcionales del Sistema

Para que la agencia pueda llevar el control de los casos sin enredarse ni perder el rastro de las investigaciones, el sistema cubre estos requerimientos:

1. **Creación o reinicio del caso:** El programa permite registrar la información básica del caso (nombre, código de identificación y el detective a cargo). Al iniciar un caso nuevo, automáticamente se resetean las ubicaciones y las pistas.
2. **Registro de ubicaciones con posición fija:** Permite guardar una nueva ubicación solicitando explícitamente el índice (del 0 al 4) en un arreglo de tamaño fijo (máximo 5). Verifica que la posición esté disponible y en el rango.
3. **Consulta general de ubicaciones:** Permite recorrer el arreglo e imprimir solo aquellas casillas que tengan una ubicación guardada, ignorando por espacios que están en null.
4. **Consulta puntual de una ubicación:** El usuario puede meter un número de casilla para ver qué hay. Si está vacía o el número no existe dentro del rango, el programa avisa sin detenerse.
5. **Modificación de riesgo y estado:** Se puede actualizar el nivel de riesgo (entre 1 y 10) y el estado de una ubicación anteriormente.
6. **Descarte de ubicaciones:** Si una ubicación deja de ser importante, el programa permite descartarla asignando nuevamente un valor null a su casilla dentro del arreglo.
7. **Registro dinámico de pistas:** Permite meter todas las pistas que vayan saliendo durante la investigación dentro de un ArrayList. Antes de registrarla, valida que el código no esté repetido.
8. **Consulta general de pistas:** Despliega todo el listado de evidencias guardadas en el ArrayList. Si no hay ninguna, el programa avisara.
9. **Búsqueda de pista por código:** Recorre el ArrayList buscando coincidencia por el código único de la evidencia.

10. **Modificación de pistas:** Permite cambiar la descripción, tipo de evidencia, importancia y confiabilidad de una pista existente.
11. **Eliminación de pistas:** Si una pista resulta ser falsa o irrelevante, se busca por su código y se remueve directamente del ArrayList.
12. **Generación del reporte de investigación:** El sistema calcula y muestra en pantalla la cantidad de ubicaciones usadas y disponibles, la ubicación más riesgosa, el total de pistas, la pista más importante, la pista más confiable y el promedio general de importancia.
13. **Manejo defensivo de errores:** El programa sobrevive a letras ingresadas cuando se pedían números (InputMismatchException), a rangos numéricos inválidos (IllegalArgumentException) y a consultas fuera del tamaño del arreglo (IndexOutOfBoundsException).

2. Propósito de cada Clase Identificada

- **Ubicacion:** Es el “molde” para representar cada sitio físico de la investigación. Su único propósito es almacenar la información de un lugar (código, nombre, dirección, riesgo y estado) y asegurarse de que el nivel de riesgo siempre esté dentro del rango permitido (1 a 10).
- **Pista:** Esta clase modela cada evidencia encontrada durante el caso. Se encarga de guardar sus datos básicos y de hacer valer las reglas del negocio para sus métricas: la importancia tiene que estar entre 1 y 10, y la confiabilidad entre 0 y 100.
- **CasoPolicial:** Es la clase cerebro o controladora del modelo. Guarda los datos generales del caso y administra directamente la estructura rígida de las ubicaciones (Ubicacion[]) y la estructura dinámica de las pistas (List<Pista>). Aquí está todo lo que usamos para buscar, agregar, eliminar y calcular las estadísticas del reporte.
- **Main:** el rol principal es ser la interfaz de consola con el usuario, desplegar el menú, capturar lo que el usuario escribe mediante la clase Scanner y envolver las operaciones en bloques try-catch-finally para que el sistema nunca colapse o crashe-e

3. Propósito de los Atributos y Colecciones

Clase Ubicacion

- **codigo (String):** Un identificador corto o clave para el lugar.
- **nombre (String):** El nombre descriptivo del sitio.
- **direccion (String):** La dirección física o reseña de dónde queda el lugar.
- **nivelRiesgo (int):** Mide qué tan peligroso es el sitio (de 1 a 10).
- **estado (String):** Situación actual del lugar.

Clase Pista

- **codigo (String):** Identificador clave único para la evidencia.
- **descripcion (String):** Detalle de lo que se encontró o de qué se trata la pista.
- **tipoEvidencia (String):** Categoría de evidencia.
- **nivelImportancia (int):** Mide qué tanto aporta al caso (de 1 a 10).
- **nivelConfiableidad (int):** Porcentaje de certeza o veracidad de la pista (de 0 a 100).

Clase CasoPolicial

- **nombreCaso (String):** Título del caso investigado.
- **codigoCaso (String):** Código de referencia o expediente del caso.
- **detectiveResponsable (String):** Nombre del detective asignado.
- **ubicaciones (Ubicacion[]):** Arreglo básico de tamaño fijo de 5 posiciones. Se eligió un arreglo tradicional porque las ubicaciones máximas a investigar están definidas rígidamente desde el inicio y sus posiciones vacías se identifican directamente con null.
- **pistas (List<Pista> / ArrayList<Pista>):** Colección dinámica basada en ArrayList. Se utiliza porque a lo largo de una investigación no sabemos cuántas pistas van a aparecer; permite agregar, buscar y remover elementos de forma elástica.

4. Propósito de los Métodos e Identificación de Excepciones

Métodos en Ubicacion y Pista

- **Constructores, getters y setters:** Permiten instanciar y acceder a los atributos respetando el encapsulamiento.
- **setNivelRiesgo(int) / setNivelImportancia(int) / setNivelConfiabilidad(int):** Validan los rangos. Si el usuario mete un valor fuera de lo permitido, lanzan un `IllegalArgumentException`.

Métodos en CasoPolicial

- **registrarUbicacion(int posicion, Ubicacion ubicacion):** Valida que el índice esté entre 0 y 4 (si no, lanza `IndexOutOfBoundsException`) y que la casilla no esté ya ocupada (`!= null`), lanzando `IllegalArgumentException` si ya hay algo guardado.
- **obtenerUbicacion(int posicion):** Retorna el objeto en esa posición o `null` si está vacía.
- **descartarUbicacion(int posicion):** Asigna `null` en la casilla indicada para dejarla libre de nuevo.
- **registrarPista(Pista pista):** Verifica que el código no exista previamente en el `ArrayList`. Si está duplicado, lanza `IllegalArgumentException`.
- **buscarPista(String codigo) / eliminarPista(String codigo):** Recorren el `ArrayList` para consultar o remover la pista correspondiente.
- **Métodos de cálculo (obtenerUbicacionMayorRiesgo, obtenerPistaMayorImportancia, etc.):** Recorren las colecciones validando que no estén vacías para no tener errores de división entre cero o punteros nulos.

Métodos en Main

- **main(String[] args):** Controla el ciclo principal del programa y contiene la estructura try-catch para atrapar cualquier fallo durante la interacción.
- **Manejo de InputMismatchException:** Atrapa cuando el usuario mete texto en vez de enteros al usar el Scanner, limpia el buffer con scanner.nextLine() y permite continuar navegando en el menú sin que el programa se cierre.
- **Uso de finally:** Incluye un bloque finally al final del ciclo para garantizar la impresión de una línea separadora o registro de fin de acción, asegurando su ejecución sin importar si la operación tuvo éxito o si saltó una excepción.

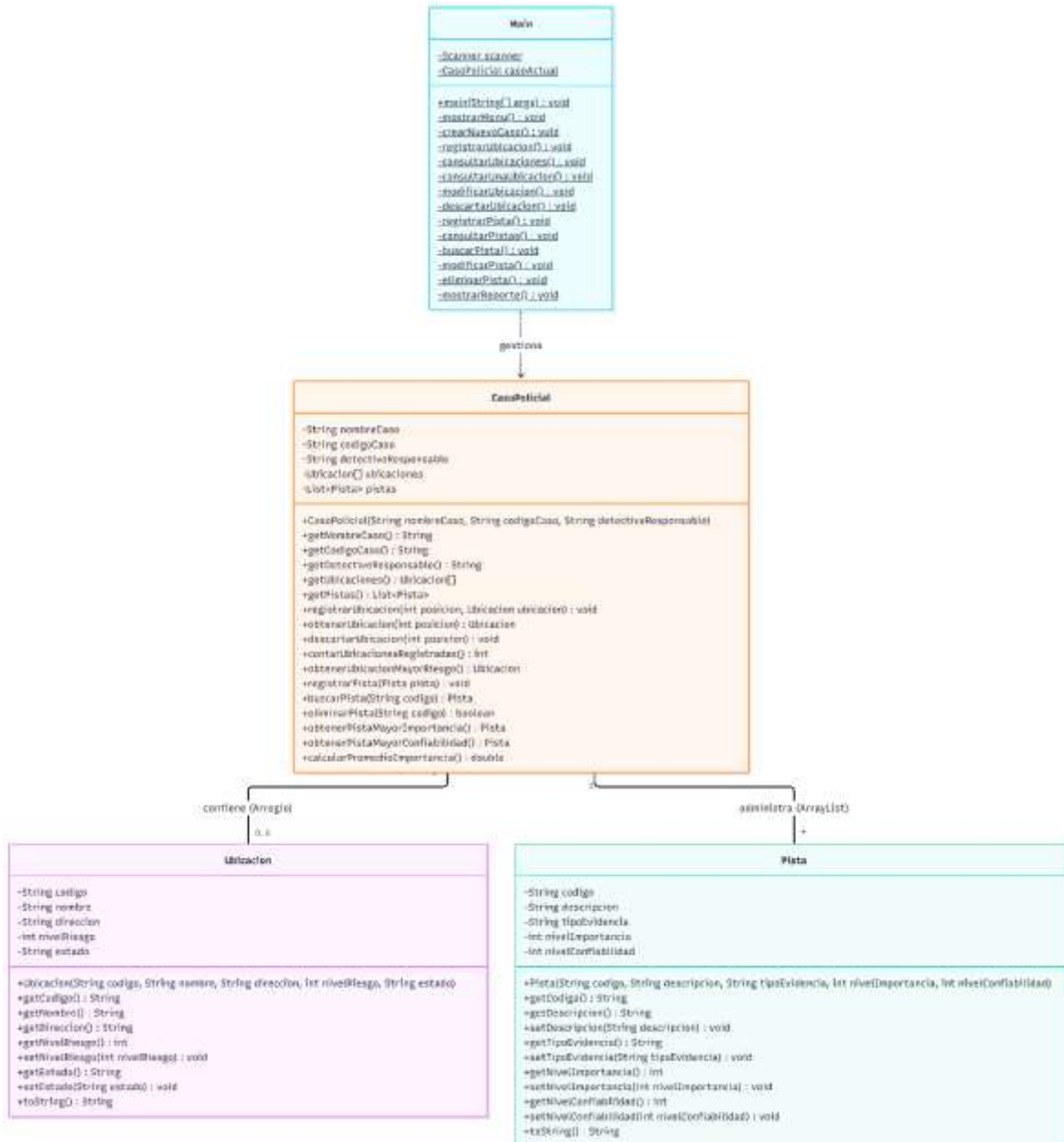
5. Respuestas a las Preguntas Guía del Análisis

- **¿Qué propiedades y métodos tendrá cada clase?** Las propiedades y métodos están definidos en los apartados 3 y 4. Ubicacion y Pista guardan datos específicos y sus validaciones; CasoPolicial concentra las colecciones y la lógica de negocio; y Main lleva la interacción con la consola.
- **¿Qué tipo deben tener las propiedades y métodos de cada clase?** Los atributos descriptivos son de tipo String, las escalas y métricas son int, el arreglo básico es Ubicacion[] y el listado dinámico es List<Pista>. Los métodos retornan tipos primitivos (boolean, int, double), referencias a objetos (Ubicacion, Pista) o void si solo realizan una acción.

- **¿Cuál de las propiedades identificadas debe implementarse utilizando un arreglo básico? ¿Qué tipo de objetos almacenará y cuál será su tamaño?** La propiedad ubicaciones de la clase CasoPolicial. Almacena objetos de tipo Ubicacion y tiene un tamaño fijo de 5 elementos.
- **¿Cuál de las propiedades identificadas debe implementarse utilizando un ArrayList? ¿Qué tipo de objetos almacenará?** La propiedad pistas de la clase CasoPolicial. Almacena objetos de tipo Pista de forma dinámica.
- **¿Cuáles deben ser los modificadores de visibilidad de los miembros en cada clase?** Todos los atributos son **privados (private)** para no se pueda cambiar el encapsulamiento. Los métodos, constructores y constantes accesibles desde fuera son **públicos (public)**, a excepción de los métodos auxiliares del menú en Main que están como **private static**.
- **¿Qué parámetros serán requeridos por los métodos en sus clases?** Los setters reciben el nuevo valor a asignar; los métodos de registro reciben los índices o las instancias creadas (posicion, Ubicacion, Pista); y los métodos de búsqueda y eliminación reciben los códigos String o posiciones numéricas.
- **¿Cómo proveerá de valores iniciales a sus objetos? ¿Qué valores deberán validarse antes de modificar el estado de los objetos?** Los valores iniciales se pasan a través de los constructores de cada clase. Antes de actualizar el estado, se validan los rangos numéricos (riesgo 1-10, importancia 1-10, confiabilidad 0-100) y la disponibilidad de espacio en el arreglo.

- **¿Cómo determinará si una posición del arreglo contiene una ubicación o contiene null?** Se evalúa directamente con la condición `if (ubicaciones[posicion] != null)`. Si es diferente de null, hay una ubicación válida; si no, la casilla se considera disponible.
- **¿Cómo realizará las operaciones de búsqueda, modificación y eliminación dentro del ArrayList?** mediante iteraciones (bucles `for` o `for-each`) comparando el código buscado con `p.getCodigo().equalsIgnoreCase(codigo)`. Al encontrar la coincidencia, se leen/actualizan sus atributos mediante setters o se elimina el objeto usando el método `.remove()` del ArrayList.
- **¿Qué situaciones del programa pueden producir excepciones? Identifique qué excepciones deberán manejarse y en qué partes del programa utilizará try-catch y finally.**
 - `InputMismatchException`: Ocurre en Main cuando se ingresan caracteres no numéricos en lecturas de Scanner. Se captura en el bloque try-catch del menú y se limpia el buffer.
 - `IllegalArgumentException`: Se produce en Ubicacion y Pista al ingresar rangos fuera de límite, o en CasoPolicial al intentar registrar pistas con códigos duplicados o en casillas ocupadas.
 - `IndexOutOfBoundsException`: Se genera al intentar acceder a posiciones menores a 0 o mayores a 4 en el arreglo de ubicaciones.
 - El bloque finally se ubica dentro de la estructura principal de control del menú en Main para ejecutar acciones de limpieza o formateo después de procesar cada opción.

Diagrama UML



Repositorio De GIT:

<https://github.com/Boxman215/-Arreglos-y-excepciones-C-digo.git>